

2026-05-05-p1-f13-lsp-integration

P1-F13 — LSP Integration Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Bring real Language-Server Protocol diagnostics into HelixCode for 5 curated servers (gopls, rust-analyzer, pyright, typescript-language-server, clangd). Lazy-spawn + 5-minute idle timeout per server. Auto-trigger after every successful `fs_edit / fs_write / multi_edit_commit` and inline the diagnostic summary into the tool result. Full user surface: `/lsp slash + helixcode lsp cobra (status / restart / list-servers / stop)`.

Architecture: New `internal/tools/`
`{lsp_types,lsp_client,lsp_manager,lsp_servers,lsp,lsp_fakeserver}.go`, plus `internal/commands/lsp_command.go` and `cmd/cli/lsp_cmd.go`. Modify `internal/tools/registry.go` to register the two LSP tools, accept an `LSPManager` via `SetLSPManager`, and post-Execute call `lspManager.NotifyChange` for edit-class tools (attaching `lsp_diagnostics` to the tool result). Modify `cmd/cli/main.go` to construct + wire + close the manager. Two new external deps: `go.lsp.dev/jsonrpc2` and `go.lsp.dev/protocol` (canonical Go LSP libraries).

Tech Stack: Go 1.26, testify v1.11, spf13/cobra v1.8 — all already present. **Two NEW external deps:** - `go.lsp.dev/jsonrpc2 v0.10.0` — JSON-RPC 2.0 framing over arbitrary `io.ReadWriter`. Justification: hand-rolling the framing layer is exactly the class of work where bluffs hide; this is the canonical library used by gopls itself. - `go.lsp.dev/protocol v0.12.0` — typed LSP protocol structs (`InitializeParams`, `DidOpenTextDocumentParams`, `PublishDiagnosticsParams`, etc.). Justification: avoids re-defining ~200 LSP types by hand; the upstream library is the source of truth and pulls cleanly into Go 1.26.

Spec: `docs/superpowers/specs/2026-05-05-p1-f13-lsp-integration-design.md` (commit ed36237)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F13 surface):

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
  internal/tools/lsp.go internal/tools/lsp_client.go internal/
  tools/lsp_manager.go \
  internal/tools/lsp_servers.go internal/tools/lsp_types.go
  internal/tools/lsp_fakeserver \
  internal/commands/lsp_command.go cmd/cli/lsp_cmd.go && echo
  BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — most LSP servers will NOT be installed on the test machine. The Challenge MUST present TWO clearly separated sections: `MANAGER PIPELINE` (always runs, in-tree fake server) and `REAL LANGUAGE SERVERS` (gated). The fake server is a real OS subprocess speaking real LSP-framed JSON-RPC over `stdio` — NOT a Go in-process stub. Real-server tests skip with `SKIP-OK: P1-F13 <binary> not installed (install: <hint>)` per server. Bare skips break `make no-silent-skips`.

Task list



P1-F13-T01 — bootstrap evidence + advance `PROGRESS` to F13



P1-F13-T02 — `go.mod`: add `go.lsp.dev/jsonrpc2 v0.10.0 + go.lsp.dev/protocol v0.12.0` (TDD: failing import test → run; add deps; run again)



P1-F13-T03 — `internal/tools/lsp_types.go`: `Diagnostic` + `DiagnosticSummary` + `DiagnosticSeverity` + `LSPServerSpec` + `ServerStatus` (TDD)



P1-F13-T04 — `internal/tools/lsp_client.go`: `jsonrpc2` client wrapper, `initialize/shutdown` handshake, `didOpen/didChange/publishDiagnostics` (TDD with paired in-memory pipes)



P1-F13-T05 — `internal/tools/lsp_manager.go`: `lazy-spawn` + `idle-timeout` + `file-extension router` + `crash-recovery` (TDD with in-tree fake server subprocess)



P1-F13-T06 — `internal/tools/lsp_servers.go`: `curated allowlist` + `Detect` via `exec.LookPath` (TDD)



P1-F13-T07 — `internal/tools/lsp.go`: `LSPGetDiagnosticsTool` + `LSPAnalyzeDiagnosticTool` (TDD)



P1-F13-T08 — `internal/tools/registry.go`: `SetLSPManager` + `post-Execute` auto-trigger for `fs_edit/fs_write/multi_edit_commit` (TDD)



P1-F13-T09 — `/lsp slash command` + `internal/commands/lsp_command.go` (TDD)



P1-F13-T10 — `helixcode lsp cobra`: `cmd/cli/lsp_cmd.go` + `main.go` wiring + integration test (gated, `SKIP-OK` on missing servers)



P1-F13-T11 — Challenge with runtime evidence (in-tree fake server harness + real-server gated phase)



P1-F13-T12 — Feature 13 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F13 evidence section header (spec ed36237), update PROGRESS current focus to F13, insert F13 task list (12 items) after F12's. Commit docs (P1-F13-T01): bootstrap Phase 1 / Feature 13 evidence + advance PROGRESS.

Task 2: go.mod — add LSP deps (TDD)

Files: helix_code/go.mod, helix_code/go.sum, new helix_code/internal/tools/lsp_deps_smoke_test.go.

Failing test FIRST (proves the deps are needed and pinned):

```
package tools

import (
    "testing"

    "go.lsp.dev/jsonrpc2"
    "go.lsp.dev/protocol"
)

func TestLSPDeps_Smoke(t *testing.T) {
    var _ jsonrpc2.Conn          // must compile against v0.10.0+
    var _ protocol.InitializeParams
    var _ protocol.PublishDiagnosticsParams
    if jsonrpc2.ErrIdleTimeout == nil {
        t.Fatal("jsonrpc2 not wired")
    }
}
```

Run `go test ./internal/tools/...` — fails with cannot find module providing package. Then `go get go.lsp.dev/jsonrpc2@v0.10.0 go.lsp.dev/protocol@v0.12.0 && go mod tidy`. Re-run; passes. Verify both modules appear in `go.mod` with the pinned versions.

Subject: feat(P1-F13-T02): add go.lsp.dev/jsonrpc2 v0.10.0 + go.lsp.dev/protocol v0.12.0.

Task 3: lsp_types.go (TDD)

Files: new helix_code/internal/tools/lsp_types.go, new helix_code/internal/tools/lsp_types_test.go.

Define `Diagnostic`, `DiagnosticSummary`, `DiagnosticSeverity`, `DiagnosticRange`, `Position`, `LSPServerSpec`, `ServerStatus`. Mirror spec §3.3 exactly. Add a `Diagnostic.ComputeID()` helper that hashes (`FilePath|Range|Source|Code|Message`) into a stable string for `LSPAnalyzedDiagnostic` lookup.

Test:

```
func TestDiagnosticSummary_CountingBySeverity(t *testing.T) {
    sum := DiagnosticSummary{Diagnostics: []Diagnostic{
        {Severity: SeverityError},
        {Severity: SeverityError},
        {Severity: SeverityWarning},
        {Severity: SeverityInformation},
        {Severity: SeverityHint},
    }}
    sum.Recount()
    require.Equal(t, 2, sum.TotalErrors)
    require.Equal(t, 1, sum.TotalWarnings)
    require.Equal(t, 1, sum.TotalInformation)
    require.Equal(t, 1, sum.TotalHints)
}

func TestDiagnostic_ComputeID_Stable(t *testing.T) {
    d := Diagnostic{FilePath: "/a/b.go", Source: "gopls", Code:
        "E1", Message: "oops",
        Range: DiagnosticRange{Start: Position{1, 0}, End:
            Position{1, 5}}}
    require.Equal(t, d.ComputeID(), d.ComputeID())
    other := d
    other.Message = "different"
    require.NotEqual(t, d.ComputeID(), other.ComputeID())
}
```

Subject: feat(P1-F13-T03): LSP type definitions (Diagnostic, DiagnosticSummary, ServerStatus).

Task 4: lsp_client.go (TDD with paired in-memory pipes)

Files: new helix_code/internal/tools/lsp_client.go, new helix_code/internal/tools/lsp_client_test.go.

LSPClient is a jsonrpc2.Conn wrapper. Constructor accepts an io.ReadWriteCloser (so unit tests can hand it net.Pipe()-paired conns; production wires it to cmd.StdoutPipe() + cmd.StdinPipe()).

Methods (mirror spec §3.3):

```
func NewLSPClient(ctx context.Context, spec LSPServerSpec, root
    string, rw io.ReadWriteCloser) (*LSPClient, error)
func (c *LSPClient) DidOpen(ctx context.Context, path string,
    content []byte) error
func (c *LSPClient) DidChange(ctx context.Context, path string,
    content []byte) error
func (c *LSPClient) DidClose(ctx context.Context, path string)
    error
```

```

func (c *LSPClient) GetDiagnostics(path string) []Diagnostic
func (c *LSPClient) WaitForDiagnostics(ctx context.Context, path
    string, timeout time.Duration) []Diagnostic
func (c *LSPClient) Status() ServerStatus
func (c *LSPClient) Close(ctx context.Context) error

```

The `initialize` request is sent on construction; `initialized` notification follows; the `publishDiagnostics` handler is registered via the `jsonrpc2` handler interface and writes into `c.diagnostics[uri]` under `c.mu`.

Tests use `net.Pipe()` to wire the client to a goroutine that fakes the server side — the test reads outbound JSON, asserts on it, and writes back canned LSP responses.

Test:

```

func TestLSPClient_InitializeHandshake_SendsRootURI(t
    *testing.T) {
    cli, srv := net.Pipe()
    serverDone := make(chan struct{})
    go func() {
        defer close(serverDone)
        // read framed initialize, write framed result
        req := readLSPMessage(t, srv)
        require.Contains(t, string(req), `"rootUri":"file:///
            workspace"`)
        writeLSPMessage(t, srv,
            `{"jsonrpc":"2.0","id":1,"result":{"capabilities":{}}}`)
        // consume the `initialized` notification
        _ = readLSPMessage(t, srv)
    }()
    c, err := NewLSPClient(context.Background(),
        LSPServerSpec{Name: "fake", Binary: "fake"}, "/"
            workspace",
        rwc{cli})
    require.NoError(t, err)
    defer c.Close(context.Background())
    <-serverDone
}

func TestLSPClient_DidChange_VersionIncrement(t *testing.T) { /*
    version 1 then 2 */ }
func TestLSPClient_PublishDiagnostics_StoredByURI(t *testing.T)
    { /* server pushes diag, client surfaces */ }
func TestLSPClient_WaitForDiagnostics_Timeout(t *testing.T) { /*
    no diag arrives, returns empty */ }
func TestLSPClient_Shutdown_Graceful(t *testing.T) { /* shutdown
    + exit observed in order */ }

```

Subject: feat(P1-F13-T04): LSPClient jsonrpc2 wrapper with `initialize`/`didOpen`/`didChange`/`publishDiagnostics`.

Task 5: lsp_manager.go (TDD with in-tree fake server subprocess)

Files: new helix_code/internal/tools/lsp_manager.go, new helix_code/internal/tools/lsp_manager_test.go, new helix_code/internal/tools/lsp_fakeserver/main.go.

The fake server (lsp_fakeserver/main.go) is a tiny standalone Go program that: - reads LSP-framed JSON-RPC from stdin, - responds to initialize with a minimal capabilities result, - on textDocument/didOpen and textDocument/didChange publishes a deterministic diagnostic at line 0 col 0 ("p1f13-fake-diagnostic"), - on shutdown returns null, on exit calls os.Exit(0).

The manager test builds it once via `go build -o $TMPDIR/p1f13_fake ./internal/tools/lsp_fakeserver` in TestMain, then runs LSPManager against a synthetic spec {Binary: \$TMPDIR/p1f13_fake, Extensions: [".fake"]}. Real subprocess, real stdio, real JSON-RPC.

LSPManager API (spec §3.3):

```
func NewLSPManager(opts LSPManagerOptions) *LSPManager
func (m *LSPManager) NotifyChange(ctx context.Context, absPath string) (*DiagnosticSummary, error)
func (m *LSPManager) GetDiagnostics(ctx context.Context, absPath string, minSeverity DiagnosticSeverity) (*DiagnosticSummary, error)
func (m *LSPManager) Status() []ServerStatus
func (m *LSPManager) Restart(ctx context.Context, name string) error
func (m *LSPManager) Stop(ctx context.Context, name string) error
func (m *LSPManager) Close(ctx context.Context) error
```

Tests:

```
func TestLSPManager_LazySpawn_OnFirstFakeFile(t *testing.T) {
    bin := buildFakeServer(t)
    m := NewLSPManager(LSPManagerOptions{Root: t.TempDir(),
        Specs: []LSPServerSpec{{Name: "fake", Binary: bin,
            Extensions: []string{".fake"}}},
        IdleTimeout: time.Minute})
    defer m.Close(context.Background())
    require.Empty(t, m.Status())
    p := writeFile(t, ".fake", "anything")
    summary, err := m.NotifyChange(context.Background(), p)
    require.NoError(t, err)
    require.NotNil(t, summary)
    require.GreaterOrEqual(t,
        summary.TotalErrors+summary.TotalWarnings, 1)
    require.Len(t, m.Status(), 1)
```

```

}

func TestLSPManager_UnknownExtension_NoSpawn_NoError(t
    *testing.T) { /* .xyz → empty summary */ }
func TestLSPManager_BinaryNotOnPath_NoError(t *testing.T) { /*
    Detect returns err → empty summary */ }
func TestLSPManager_IdleTimeout_ReapsServer(t *testing.T) { /*
    IdleTimeout=200ms, sleep 500ms, Status empty */ }
func TestLSPManager_CrashRecovery_Restarts(t *testing.T) { /*
    SIGKILL the subprocess; next NotifyChange respawns */ }
func TestLSPManager_Close_AllSubprocessesReaped(t *testing.T) { /
    * PIDs gone */ }

```

Subject: feat(P1-F13-T05): LSPManager with lazy spawn, idle timeout, file-extension router, crash recovery + in-tree fake LSP server.

Task 6: lsp_servers.go — curated allowlist (TDD)

Files: new helix_code/internal/tools/lsp_servers.go, new helix_code/internal/tools/lsp_servers_test.go.

```

func DefaultLSPServerSpecs() []LSPServerSpec {
    return []LSPServerSpec{
        {Name: "gopls", Binary: "gopls", Extensions:
            []string{".go"}},
        {Name: "rust-analyzer", Binary: "rust-analyzer",
            Extensions: []string{".rs"}},
        {Name: "pyright", Binary: "pyright-langserver", Args:
            []string{"--stdio"}, Extensions: []string{".py"}},
        {Name: "typescript-language-server", Binary: "typescript-
            language-server", Args: []string{"--stdio"},
            Extensions: []string{".ts", ".tsx", ".jsx", ".js"}},
        {Name: "clangd", Binary: "clangd", Extensions:
            []string{".c", ".cpp", ".cc", ".h", ".hpp"}},
    }
}

```

```

func DetectInstalled(specs []LSPServerSpec, lookPath
    func(string) (string, error)) map[string]string

```

Tests assert (a) all 5 servers appear, (b) extensions match the spec, (c) DetectInstalled returns a map with only those whose lookPath succeeded. Use an injected lookPath so the test never depends on the host having any of them installed.

Subject: feat(P1-F13-T06): curated 5-server allowlist + Detect via exec.LookPath.

Task 7: lsp.go — agent tools (TDD)

Files: new helix_code/internal/tools/lsp.go, new helix_code/internal/tools/lsp_test.go.

Adapters that satisfy the existing Tool interface (Name/Description/Schema/Category/Validate/Execute). Both wrap an *LSPManager injected via constructor. Category constant CategoryLSP ToolCategory = "lsp" added in registry.go.

Tests use the in-tree fake server via the manager:

```
func TestLSPGetDiagnosticsTool_ReturnsSummaryFromManager(t
    *testing.T) {
    m := managerWithFakeServer(t)
    tool := &LSPGetDiagnosticsTool{manager: m}
    p := writeFile(t, ".fake", "x")
    _, _ = m.NotifyChange(context.Background(), p)
    res, err := tool.Execute(context.Background(),
        map[string]interface{}{"file_path": p})
    require.NoError(t, err)
    sum := res.(*DiagnosticSummary)
    require.GreaterOrEqual(t, len(sum.Diagnostics), 1)
}
func TestLSPGetDiagnosticsTool_FiltersBySeverity(t *testing.T)
    { /* severity=warning excludes hints */ }
func TestLSPAnalyzeDiagnosticTool_LookupByID(t *testing.T) { /*
    roundtrip: get summary, pick ID, analyze */ }
```

Subject: feat(P1-F13-T07): LSPGetDiagnostics + LSPAnalyzeDiagnostic agent tools.

Task 8: registry.go — SetLSPManager + auto-trigger (TDD)

Files: modify helix_code/internal/tools/registry.go, new helix_code/internal/tools/registry_lsp_test.go.

Add to ToolRegistry:

```
lspManager *LSPManager // optional; nil disables LSP auto-trigger

func (r *ToolRegistry) SetLSPManager(m *LSPManager) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.lspManager = m
}
```

In Execute, after a successful inner Execute and after markPlanActionExecuted, before returning:

```
if execErr == nil && r.lspManager != nil && isEditTool(name) {
    if path := extractEditedPath(name, params); path != "" {
```



```

        if summary, _ := r.lspManager.NotifyChange(ctx, path);
        summary != nil && len(summary.Diagnostics) > 0 {
            result = wrapResultWithLSP(result, summary)
        }
    }
}

```

isEditTool returns true for "fs_edit", "fs_write", "multi_edit_commit". wrapResultWithLSP ensures the result is a map[string]interface{} with key "lsp_diagnostics" set; if the inner result is not already a map, it wraps it as {"result": <inner>, "lsp_diagnostics": <summary>}. Also register LSPGetDiagnosticsTool and LSPAnalyzeDiagnosticTool in registerAllTools() (gated: only when lspManager != nil, registered lazily from SetLSPManager).

Tests:

```

func TestRegistry_AutoTriggerAfterFSEdit_AttachesLSPDiagnostics(t
    *testing.T) { /* fs_edit then assert
        result["lsp_diagnostics"] populated */ }
func TestRegistry_AutoTriggerSkipsOnEditError(t *testing.T) { /*
    fs_edit fails → no NotifyChange call */ }
func TestRegistry_AutoTriggerNeverBlocksOnLSPError(t *testing.T)
    { /* manager returns err → user edit still succeeds */ }
func TestRegistry_NoLSPManager_NoAutoTrigger(t *testing.T) { /*
    without SetLSPManager, edits succeed unchanged */ }
func TestRegistry_LSPToolsRegistered(t *testing.T) { /*
    Get("LSPGetDiagnostics") succeeds after SetLSPManager
    */ }

```

Subject: feat(P1-F13-T08): registry.SetLSPManager + post-Execute auto-trigger for fs_edit/fs_write/multi_edit_commit.

Task 9: /lsp slash command (TDD)

Files: new helix_code/internal/commands/lsp_command.go, new helix_code/internal/commands/lsp_command_test.go.

Mirrors F09 commands_command.go and F11 sessions_command.go patterns.

Subcommands: - /lsp status — calls manager.Status(), renders a text/tabwriter table. - /lsp list-servers — prints DefaultLSPServerSpecs() and DetectInstalled() results side-by-side. - /lsp restart [name] — manager.Restart(ctx, name) (or all when name == ""). - /lsp stop [name] — manager.Stop(ctx, name) (or all).

Tests:

```

func TestLSPCommand_StatusEmptyManager_RendersHeaderRow(t
    *testing.T) { /* no servers running → just headers */ }
func TestLSPCommand_StatusWithFakeServer_RendersOneRow(t
    *testing.T) { /* uses managerWithFakeServer helper */ }
func TestLSPCommand_ListServers_Shows5(t *testing.T) {}
func TestLSPCommand_RestartByName_DelegatesToManager(t
    *testing.T) {}
func TestLSPCommand_Stop_DelegatesToManager(t *testing.T) {}

```

Subject: feat(P1-F13-T09): /lsp slash command (status / restart / list-servers / stop).

Task 10: helixcode lsp cobra + main.go wiring + integration test

Files: new helix_code/cmd/cli/lsp_cmd.go, new helix_code/cmd/cli/lsp_cmd_test.go, modify helix_code/cmd/cli/main.go, new helix_code/tests/integration/lsp_test.go (//go:build integration).

Cobra mirrors F11 sessions_cmd.go:

```

type lspCmdDeps struct{ Manager *tools.LSPManager }
func newLSPCmd(deps lspCmdDeps) *cobra.Command { /* root + 4
    subcommands */ }

```

main.go startup wiring:

```

specs := tools.DefaultLSPServerSpecs()
lspMgr := tools.NewLSPManager(tools.LSPManagerOptions{
    Root: workspaceRoot, Specs: specs, IdleTimeout: 5 *
        time.Minute,
})
defer lspMgr.Close(context.Background())
registry.SetLSPManager(lspMgr)
slashRegistry.Register(commands.NewLSPCommand(lspMgr))
rootCmd.AddCommand(newLSPCmd(lspCmdDeps{Manager: lspMgr}))

```

Integration test (gated, per spec §5.2):

```

//go:build integration
// +build integration

func TestLSP_GoplsRoundTrip(t *testing.T) {
    if _, err := exec.LookPath("gopls"); err != nil {
        t.Skip("SKIP-OK: P1-F13 gopls not installed (install: go
            install golang.org/x/tools/gopls@latest)")
    }
    dir := t.TempDir()
    src := filepath.Join(dir, "broken.go")

```

```

require.NoError(t, os.WriteFile(src, []byte("package
    mismatch\nfunc {\n"), 0644))
m := tools.NewLSPManager(tools.LSPManagerOptions{Root: dir,
    Specs: tools.DefaultLSPServerSpecs(), IdleTimeout: 30 *
    time.Second})
defer m.Close(context.Background())
sum, err := m.NotifyChange(context.Background(), src)
require.NoError(t, err)
require.NotNil(t, sum)
require.Greater(t, sum.TotalErrors, 0, "expected gopls to
    surface a syntax error")
}
// Plus TestLSP_RustAnalyzerRoundTrip, TestLSP_PyrightRoundTrip,
// TestLSP_TypescriptLanguageServerRoundTrip,
// TestLSP_ClangdRoundTrip
// each with their own SKIP-OK + install hint per spec §5.2.
// Plus TestLSP_AutoTriggerThroughFSEdit_GoplsEndToEnd (gated on
// gopls).

```

Subject: feat(P1-F13-T10): wire LSPManager into main.go + helixcode
 lsp cobra + gated integration tests.

Task 11: Challenge with runtime evidence

Files: new helix_code/tests/integration/cmd/p1f13_challenge/main.go,
 new challenges/p1-f13-lsp-integration/CHALLENGE.md, new challenges/
 p1-f13-lsp-integration/run.sh.

The harness builds the in-tree fake LSP server, then runs two phases. Output skeleton:

```

=== MANAGER PIPELINE (always runs, in-tree fake server) ===
[PASS] manager: lazy spawn on first .fake file
[PASS] manager: file-extension router routes .fake to fake-server only
[PASS] manager: unknown extension returns empty summary, no error
[PASS] manager: binary-not-on-PATH returns empty summary, no error
[PASS] manager: idle timeout reaps subprocess after 200ms
[PASS] manager: crash recovery respawns subprocess after SIGKILL
[PASS] client: initialize handshake (rootURI verified)
[PASS] client: didOpen + didChange version increment
[PASS] client: publishDiagnostics stored by URI
[PASS] auto-trigger: fs_edit attaches lsp_diagnostics to result
[PASS] auto-trigger: never blocks user edit on LSP error

=== REAL LANGUAGE SERVERS (gated) ===
[PASS|skipped: gopls not on PATH (install: go install golang.org/x/tools/g
[PASS|skipped: rust-analyzer not on PATH (install: rustup component add ru
[PASS|skipped: pyright not on PATH (install: npm i -g pyright)]
[PASS|skipped: typescript-language-server not on PATH (install: npm i -g t
[PASS|skipped: clangd not on PATH (install: apt install clangd OR brew ins

```

SUMMARY: MANAGER=11/11 PASS; REAL_SERVERS=<n>/5 PASS (<5-n> skipped)

The Challenge MUST NOT print [PASS] for a server that wasn't actually invoked. Anti-bluff smoke clean check appended to the harness output. Verbatim output captured into 06_phase_1_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F13-T11): challenge with runtime evidence (in-tree fake server pipeline + gated real servers).

Task 12: Close-out + push

Tick all 12 items in PROGRESS, advance PROGRESS focus to F14 candidate, run final verification (make verify-compile, anti-bluff smoke, go test -count=1 ./internal/tools/... ./internal/commands/...), commit chore(P1-F13-T12): close out feature 13 – LSP integration, push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T03 types (§3.3), T04 client (§3.3, §4), T05 manager (§3.3, §4.2, §4.4, §4.5), T06 servers (§1, §3.3), T07 tools (§3.4), T08 registry auto-trigger (§4.3), T09 slash (§3.4), T10 cobra + integration (§3.4, §5.2, §6.2), T11 Challenge (§5.2, §6.3), T12 close-out (§9).
2. **TDD:** every code task starts with a failing test that exercises the real code path (paired in-memory pipes for client; real subprocess for manager; auto-trigger asserted via the registry's actual Execute path).
3. **Type consistency:** Diagnostic, DiagnosticSummary, DiagnosticSeverity, LSPServerSpec, ServerStatus, LSPManager, LSPClient, LSPGetDiagnosticsTool, LSPAnalyzedDiagnosticTool — names match across spec §3.3 and plan T03–T08.
4. **Two new external deps:** go.lsp.dev/jsonrpc2 v0.10.0 + go.lsp.dev/protocol v0.12.0. Both confirmed absent from helix_code/go.mod at spec time. Justification: these are the canonical Go LSP libraries (used by gopls itself); hand-rolling JSON-RPC framing + ~200 LSP types is exactly the bluff-prone work the Constitution asks us to avoid.
5. **Anti-bluff (§5.2):** Challenge has TWO sections; the always-runs section uses a real OS subprocess (the in-tree fake server) speaking real LSP-framed JSON-RPC over real stdio, NOT a Go in-process stub. The gated section explicitly skips with SKIP-OK: P1-F13 <binary> not installed (install: <hint>) per server.
6. **Auto-trigger never blocks the user:** r.lspManager.NotifyChange errors are logged WARN and dropped; the user's edit always succeeds. This is asserted by TestRegistry_AutoTriggerNeverBlocksOnLSPError.
7. **CONST-042:** INFO logs in LSPClient log only len(message) and basenames; full diagnostic content is DEBUG-only behind HELIX_LSP_DEBUG=1. Challenge verifies INFO log lines do not contain test-diagnostic substrings.
8. **Cross-platform:** pure Go; os/exec stdio piping behaves identically on Linux/macOS/Windows (the upstream go.lsp.dev/jsonrpc2 library is itself used by gopls on all three).
9. **Branch + push:** stays on main, non-force to all four remotes (per CONST-043); explicit user authorization is requested at T12 before pushing.

10. **Reality check:** the existing Tool interface in `internal/tools/registry.go` (Name/Description/Schema/Execute/Category/Validate — see `registry.go:24`) is fully compatible with the LSP tool adapters. No registry redesign required. The auto-trigger lives in the existing chokepoint (`ToolRegistry.Execute` at `registry.go:334`) — same place hooks fire — so we are extending an established pattern, not inventing one.